Dimensionality Reduction Approaches to Insider Threat Detection: From Principal Components to Interpretable Sparse Features

by

Suramya Raj Angdembay

A Thesis

Submitted to the Honors College of

The University of Southern Mississippi

in Partial Fulfillment

of Honors Requirements

Month Year

Approved by:

Dr. Haiyan Tian, Degree

School of Mathematics and Natural Sciences

Thesis Co-Advisor Name, Degree

School of Your School

Director Name, Degree

School of Your School

Joyce Inman, Ph.D., Dean

Honors College

ABSTRACT

Insider-threat detection requires distinguishing unusual activity from malicious activity. This thesis studies that distinction through principal component analysis, language-model surprisal, and sparse representations of a model's internal states on the synthetic CERT release 4.2 dataset. PCA scores daily activity by its reconstruction error. A language model trained on benign users scores the same kind of activity by how difficult its textual description is to predict. Separating predictions about a user's profile from predictions about their behavior improves the language model's ranking of insiders without retraining. Sparse-feature analysis then examines where selected internal features activate and how their removal changes predictions. These features contribute to behavior prediction, but the measured response does not establish an attack-specific mechanism. The main conclusion is that improving an anomaly score and explaining the model's internal computation are separate achievements: excluding profile predictions improves ranking, while profile information can still influence the behavior predictions that remain.

Keywords: Insider Threat; Dimensionality Reduction; Principal Component Analysis (PCA); Surprisal; Large Language Models; Mechanistic Interpretability; Sparse Autoencoders; One-Class Anomaly Detection

DEDICATION

Your dedication text here.

ACKNOWLEDGMENTS

Your acknowledgments begin here. Recognize advisors, faculty, funding agencies, and institutional support.

Contents

ABSTRACT **2**

DEDICATION **3**

ACKNOWLEDGMENTS **4**

LIST OF ABBREVIATIONS **10**

1 Introduction **11**

1.1 Example 1st Level Heading 11

Example 2nd Level Heading 11

2 Principal Component Analysis **12**

2.1 Principal Components: Motivation and Formulation 12

2.2 Inverse PCA and Reconstruction Error 14

2.3 A Worked Example and Its Geometry 15

2.4 Higher Dimensions and Anomaly Scoring 16

2.5 The CERT Dataset and Insider Cases 21

2.6 PCA Experiment: Preparation and Results 23

2.7 Summary 25

3 Language-Model Surprisal and Its Decomposition **27**

3.1 From Reconstruction to Prediction 27

3.2 From Activations to Token Probabilities 29

3.3 Surprisal and Benign-Only Adaptation 31
3.4 Day Scores and the Profile–Behavior Decomposition 32
3.5 Experimental Design 34
3.6 Results and the Connection to Interventions 36

4 Interpretable Dimensionality Reduction of the Fine-Tuned Model 38

4.1 Overview and Motivation 38
4.2 Hidden States and the Adaptation Difference 38
4.3 A Sparse Dictionary and a Worked Example 39
4.4 Choosing the Features to Test 40
4.5 Where Features Activate 41
4.6 Causal Test: Removing Features and Re-Scoring 42
4.7 Corrected Mechanistic Results 43
4.8 Limitations and Conclusions 44

A Supporting Mathematics for Language-Model Scoring 46

A.1 Why Additive Surprise Has a Logarithmic Form 46
A.2 Surprisal Written in Terms of Logits 47
A.3 Maximum Likelihood in a Frequency Model 48
A.4 Why Subtracting Class Means Gives a Different Score 49
A.5 User AUC and Equal-Insider Fold Averages 50
A.6 Normalization and Causal Attention 51
A.7 Complete Activity Record and Representation Details 52
A.8 Intervention Policies and Causal Abstraction 53

REFERENCES 55

List of Tables

2.1 Rank-1 reconstruction of the four observations. 15

2.2 The three insider scenarios of CERT release 4.2, following the answer key. . . 22

2.3 CERT feature groups and training variability 24

2.4 PCA results for all 70 insiders 25

3.1 An illustrative profile–behavior ranking change 34

3.2 User-disjoint splits of CERT release 4.2: no user belongs to more than one group. 35

3.3 Corrected language-model score decomposition 36

4.1 Sparse-feature activation location 42

4.2 Corrected feature-removal effects 43

List of Figures

2.1 Illustration of PCA in two dimensions: PCA rotates the coordinate system so that PC1 captures the largest variance while PC2 captures the smallest. Projecting onto PC1 yields a one-dimensional representation with minimal loss when PC2 contributes little variance. 16

2.2 Illustration of PCA in three dimensions: PC1, PC2, and PC3 form an orthonormal basis with decreasing explained variance. Dropping PC3 projects data onto a best-fit plane. 17

2.3 A high-dimensional PCA illustration (17 features): PCA reveals dominant axes of variation among observations (countries). Large separation along PC1 indicates an observation that differs substantially along the primary pattern in the data. 18

2.4 Synthetic two-dimensional dataset with principal component directions drawn from the center of the data. 19

2.5 Dataset represented in the PCA coordinate system where each observation is described by its PC1 and PC2 coordinates. 20

2.6 Values of the first principal component for all observations. 20

2.7 Values of the second principal component for all observations. 21

2.8 PCA ROC on all 70 insiders and 79 held-out benign users, using the 309 varying features. The marked threshold flags 35 insiders and 8 benign users. The curve is recomputed from the saved user scores of the original PCA run; no PCA refit is involved. 26

3.1 Prediction architecture and intervention location 28

LIST OF ABBREVIATIONS

APA	American Psychological Association
AUC	Area Under the (Receiver Operating Characteristic) Curve
CERT	CERT Division of the Software Engineering Institute
LoRA	Low-Rank Adaptation
PCA	Principal Component Analysis
QLoRA	Quantized Low-Rank Adaptation
ROC	Receiver Operating Characteristic
SAE	Sparse Autoencoder
USM	The University of Southern Mississippi

Chapter I

Introduction

Begin the first chapter here. Indent the first line of each paragraph.

1.1 Example 1st Level Heading

Text for first-level heading.

Example 2nd Level Heading

Text for second-level heading.

Example 3rd Level Heading (In Paragraph)

Example 3rd Level Heading. Continue paragraph text immediately after the bold heading.

Chapter II

Principal Component Analysis

2.1 Principal Components: Motivation and Formulation

Principal Component Analysis (PCA) is a classical technique for identifying orthogonal directions along which the data exhibits the greatest variance. In applications such as behavioral cybersecurity logs, the raw feature space can be high-dimensional and strongly correlated, which complicates visualization, statistical modeling, and anomaly detection. PCA addresses these issues by constructing an orthonormal coordinate system whose axes (the *principal components*) are ordered by the amount of variance they explain. This chapter presents the mathematical formulation of PCA in the general p-dimensional setting, the inverse transformation that reconstructs data from the reduced space, and the reconstruction error that motivates PCA-based anomaly scoring, followed by low- and high-dimensional illustrations. Historically, the geometric origins of PCA trace back to Pearson, while the familiar algebraic eigenvalue formulation was developed by Hotelling [1, 2, 3].

PCA represents each observation as a numerical vector in $\mathbb{R}^p$ and approximates the data with a linear subspace. Its variance criterion summarizes dominant variation; it does not guarantee that high-variance directions are the most useful ones for detecting anomalies. Euclidean distance must also be meaningful in the chosen feature space. Because PCA is sensitive to the relative magnitude of features, the features are standardized first.

Let $X \in \mathbb{R}^{n \times p}$ be a data matrix with n observations and p features, whose ith row is the observation $\mathbf{x}_i^\top$. With the sample mean $\bar{\mathbf{x}} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{x}_i$ and the vector of ones $\mathbf{1} \in \mathbb{R}^n$, the centered data matrix is

$$X_c \;=\; X - \mathbf{1}\bar{\mathbf{x}}^\top,$$

which subtracts the mean from every observation. The sample covariance matrix is

$$S \;=\; \frac{1}{n-1} X_c^\top X_c \in \mathbb{R}^{p \times p}, \tag{2.1}$$

which is symmetric and positive semidefinite and therefore has an orthonormal eigendecomposition.

PCA looks for the unit direction $\mathbf{a}$ along which the projected scores $z_i = \mathbf{a}^\top(\mathbf{x}_i - \bar{\mathbf{x}})$ vary most. Their variance is $\mathbf{a}^\top S \mathbf{a}$, so the first principal direction solves

$$\max_{\mathbf{a} \in \mathbb{R}^p} \; \mathbf{a}^\top S \mathbf{a} \quad \text{subject to} \quad \mathbf{a}^\top \mathbf{a} = 1. \tag{2.2}$$

With a Lagrange multiplier λ, setting the gradient of $\mathbf{a}^\top S \mathbf{a} - \lambda(\mathbf{a}^\top \mathbf{a} - 1)$ to zero gives

$$S\mathbf{a} = \lambda \mathbf{a}. \tag{2.3}$$

The principal directions are therefore eigenvectors of S, and each eigenvalue is the variance explained along its direction [2, 3]. Ordering the eigenvalues $\lambda_1 \geq \cdots \geq \lambda_p \geq 0$ with orthonormal eigenvectors $\mathbf{u}_1, \ldots, \mathbf{u}_p$ gives $S = U \Lambda U^\top$, with $U = [\mathbf{u}_1 \;\; \cdots \;\; \mathbf{u}_p]$ and $\Lambda = \operatorname{diag}(\lambda_1, \ldots, \lambda_p)$.

To reduce the dimension to $q < p$, keep the first q directions, $U_q = [\mathbf{u}_1 \;\; \cdots \;\; \mathbf{u}_q] \in \mathbb{R}^{p \times q}$. The reduced data, or *principal component scores*, are

$$Z \;=\; X_c U_q \in \mathbb{R}^{n \times q}, \tag{2.4}$$

whose ith row is $\mathbf{z}_i^\top = (U_q^\top \mathbf{x}_{c,i})^\top$. Since the total variance is $\mathrm{tr}(S) = \sum_{j=1}^{p} \lambda_j$, the proportion of variance explained by the first q components is

$$\mathrm{PVE}(q) \;=\; \frac{\sum_{j=1}^{q} \lambda_j}{\sum_{j=1}^{p} \lambda_j}, \tag{2.5}$$

and q is commonly chosen so that $\mathrm{PVE}(q)$ exceeds a fixed threshold such as 0.95.

2.2 Inverse PCA and Reconstruction Error

PCA provides dimensionality reduction, and the *inverse PCA transformation* approximates the original data from the reduced space:

$$\widehat{X} \;=\; Z\,U_q^\top + \mathbf{1}\bar{\mathbf{x}}^\top, \qquad \widehat{\mathbf{x}}_i \;=\; U_q \mathbf{z}_i + \bar{\mathbf{x}}, \tag{2.6}$$

where Z is the reduced data, U_q holds the principal components and $\bar{\mathbf{x}}$ is the mean vector. For the centered observation $\mathbf{x}_{c,i} = \mathbf{x}_i - \bar{\mathbf{x}}$ this is $\widehat{\mathbf{x}}_{c,i} = U_q U_q^\top \mathbf{x}_{c,i}$, the orthogonal projection onto the principal subspace $\mathrm{span}\{\mathbf{u}_1, \ldots, \mathbf{u}_q\}$.

The reconstruction error measures the difference between the original and the reconstructed data. For one observation it is

$$e_i \;=\; \|\mathbf{x}_i - \widehat{\mathbf{x}}_i\|_2^2 \;=\; \|(I - U_q U_q^\top)\,\mathbf{x}_{c,i}\|_2^2, \tag{2.7}$$

the squared Euclidean distance from $\mathbf{x}_{c,i}$ to the principal subspace, and over n observations it is the mean squared error

$$\mathrm{MSE} \;=\; \frac{1}{n} \sum_{i=1}^{n} \|\mathbf{x}_i - \widehat{\mathbf{x}}_i\|_2^2. \tag{2.8}$$

A model built from normal data reconstructs normal observations well, so a large reconstruction error may indicate an anomaly. This is the basis of the PCA anomaly score used in this thesis.

2.3 A Worked Example and Its Geometry

For an illustration in the original units, consider four observations of two features,

$$X = \begin{bmatrix} 2 & 1 \\ 3 & 2 \\ 4 & 2 \\ 5 & 4 \end{bmatrix}, \qquad \bar{\mathbf{x}} = \begin{bmatrix} 3.5 \\ 2.25 \end{bmatrix}, \qquad S = \begin{bmatrix} 1.6667 & 1.5 \\ 1.5 & 1.5833 \end{bmatrix}.$$

The eigenvalues of S are $\lambda_1 \approx 3.125$ and $\lambda_2 \approx 0.125$, so the first principal component $\mathbf{u}_1 \approx (0.72,\ 0.69)$ explains about 96% of the variance. Keeping $q = 1$, the scores are $z \approx (-1.94,\ -0.53,\ 0.19,\ 2.28)$, and the inverse transformation $\widehat{\mathbf{x}}_i = z_i \mathbf{u}_1 + \bar{\mathbf{x}}$ gives the reconstructions and errors in Table 2.1.

Table 2.1: Rank-1 reconstruction of the four observations.

i	$\mathbf{x}_i$	$\widehat{\mathbf{x}}_i$	$e_i = \lVert \mathbf{x}_i - \widehat{\mathbf{x}}_i \rVert_2^2$
1	(2, 1)	(2.10, 0.91)	0.018
2	(3, 2)	(3.12, 1.88)	0.029
3	(4, 2)	(3.64, 2.38)	0.274
4	(5, 4)	(5.15, 3.82)	0.054

The third observation lies farthest from the line spanned by $\mathbf{u}_1$ and has the largest error, and $\text{MSE} = (0.018 + 0.029 + 0.274 + 0.054)/4 \approx 0.094$. The same operations apply unchanged in high dimensions.

In $\mathbb{R}^2$, PCA identifies a new pair of orthonormal axes (u_1, u_2) obtained by rotating the original coordinate system so that u_1 aligns with the direction of maximum spread. If the data are approximately distributed along a line, reducing from 2D to 1D projects the points onto the line spanned by u_1 and discards the orthogonal component (Figure 2.1). In $\mathbb{R}^3$, the

three directions (u_1, u_2, u_3) are ordered by $(\lambda_1, \lambda_2, \lambda_3)$, and reducing to 2D projects onto the plane spanned by (u_1, u_2) (Figure 2.2).

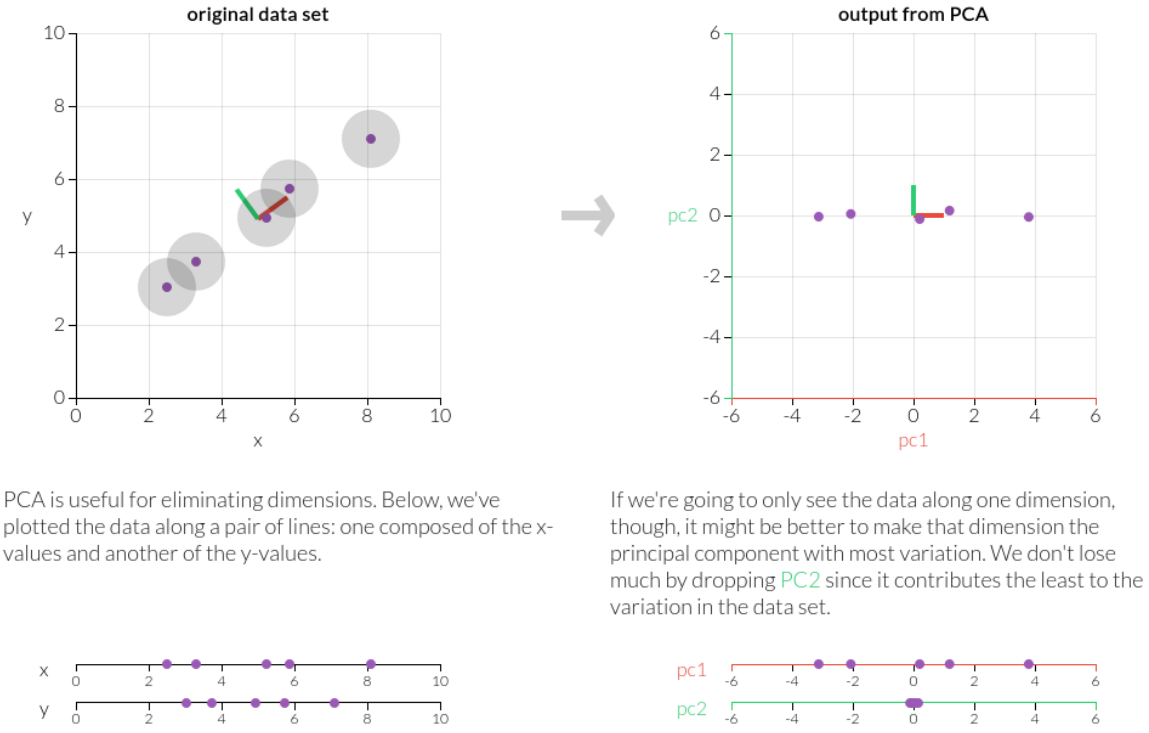


Figure 2.1: Illustration of PCA in two dimensions: PCA rotates the coordinate system so that PC1 captures the largest variance while PC2 captures the smallest. Projecting onto PC1 yields a one-dimensional representation with minimal loss when PC2 contributes little variance.

2.4 Higher Dimensions and Anomaly Scoring

PCA is often most valuable when p is large and direct visualization is not possible. To illustrate this, consider a dataset where each observation corresponds to a country and each feature corresponds to a measured quantity, producing points in $\mathbb{R}^{17}$. For example, one may treat the weekly per-person consumption of 17 food categories as a vector in $\mathbb{R}^{17}$. Although we cannot visualize $\mathbb{R}^{17}$ directly, PCA provides low-dimensional views (e.g., $q = 1$ or $q = 2$) that reveal dominant contrasts among countries.

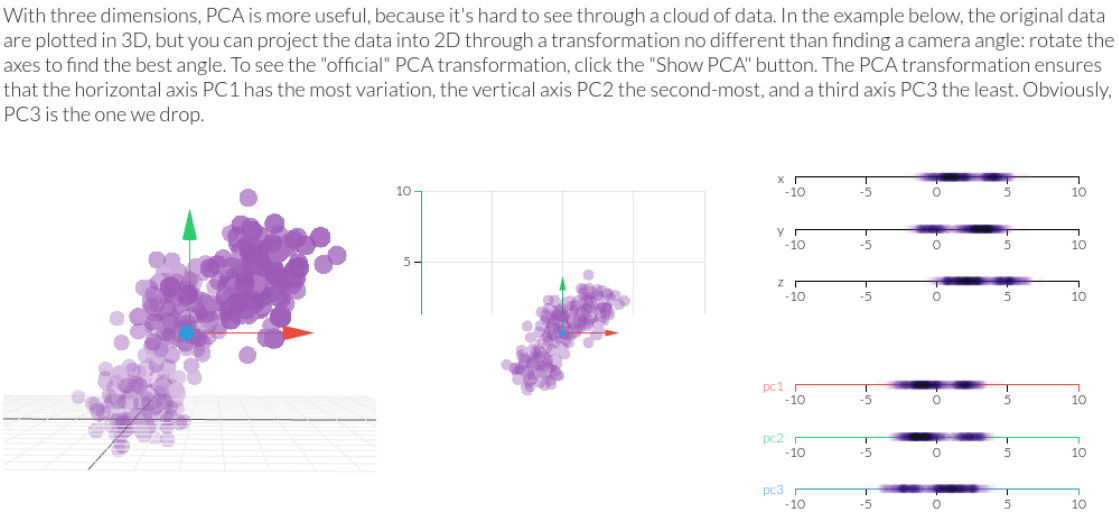


Figure 2.2: Illustration of PCA in three dimensions: PC1, PC2, and PC3 form an orthonormal basis with decreasing explained variance. Dropping PC3 projects data onto a best-fit plane.

Let $x_k \in \mathbb{R}^{17}$ denote the centered consumption vector for country k. The first PC score is $z_{k1} = u_1^\top x_k$, and the second is $z_{k2} = u_2^\top x_k$. Countries separated strongly along PC1 exhibit the greatest differences along the dominant consumption pattern, while PC2 captures the second most important pattern not explained by PC1.

A second illustration connects the geometry to anomaly scoring. To visualize how PCA organizes behavioral observations and highlights deviations from dominant patterns, we construct 30 two-dimensional points: 27 typical observations concentrated around a common trend, and three placed away from it to represent anomalous behavior. Figure 2.4 shows the data with the two principal directions drawn from its center; the first component aligns with the main trend and the second captures variation orthogonal to it. Projecting every observation onto these components gives the representation in Figure 2.5, and Figures 2.6 and 2.7 plot each observation's coordinate along the first and second components. Points that deviate from the general pattern become more clearly distinguishable in their component coordinates.

For a one-component reconstruction, an observation's error is the square of its discarded

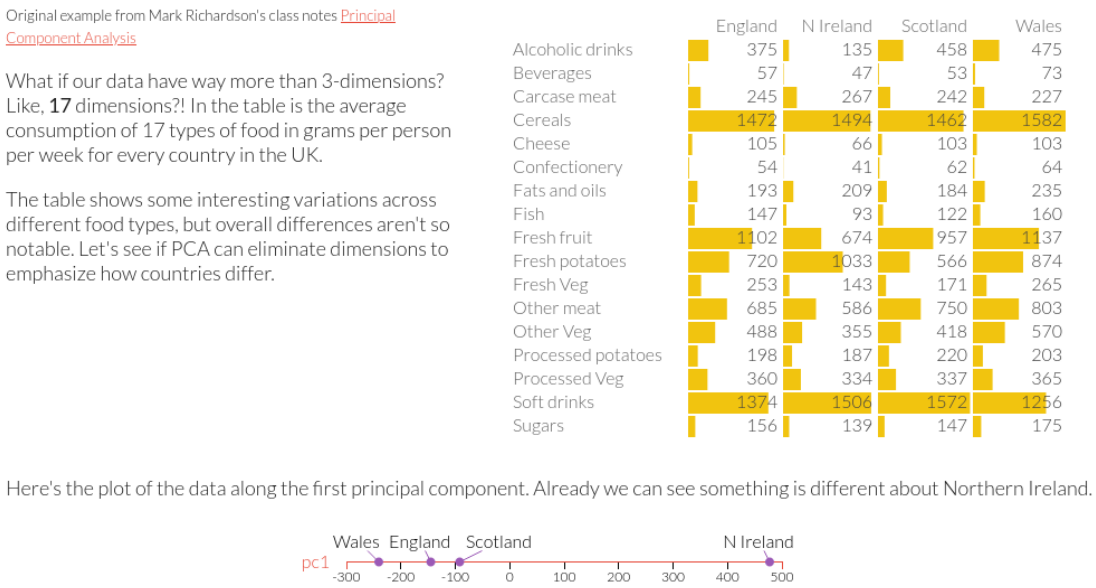


Figure 2.3: A high-dimensional PCA illustration (17 features): PCA reveals dominant axes of variation among observations (countries). Large separation along PC1 indicates an observation that differs substantially along the primary pattern in the data.

PC2 coordinate. A large PC1 coordinate alone is therefore not evidence of a large reconstruction error. These examples illustrate geometry; the CERT experiment below fits its components on benign training users only.

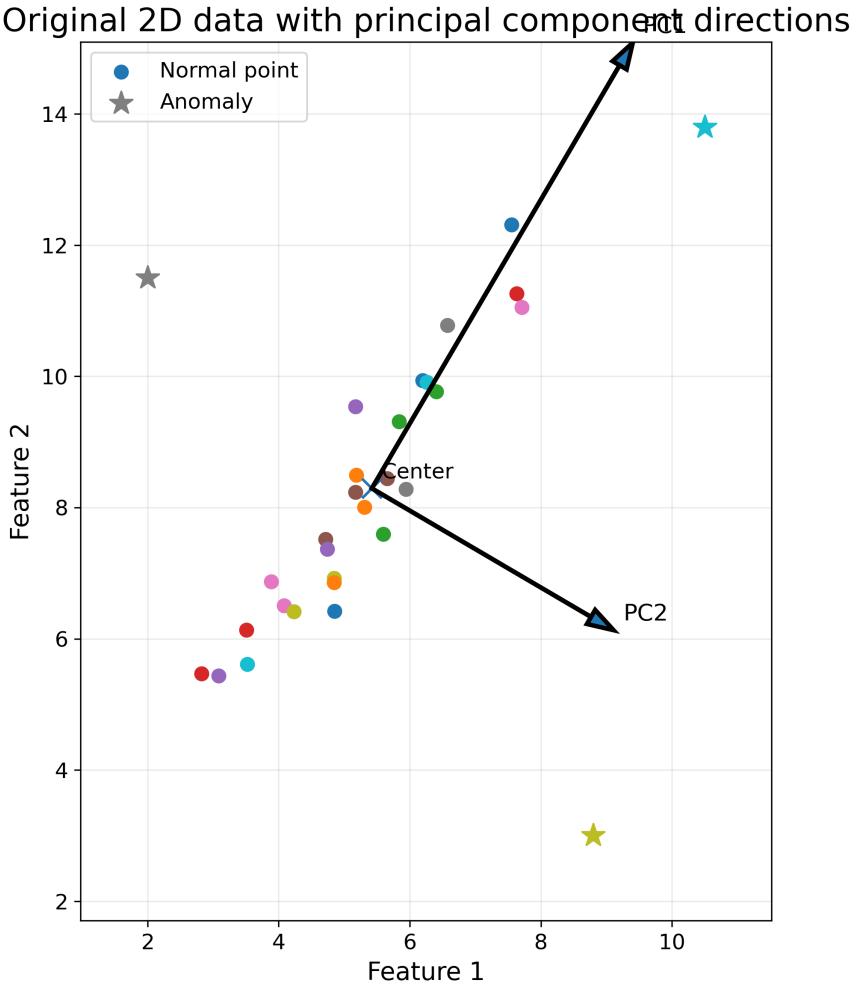


Figure 2.4: Synthetic two-dimensional dataset with principal component directions drawn from the center of the data.

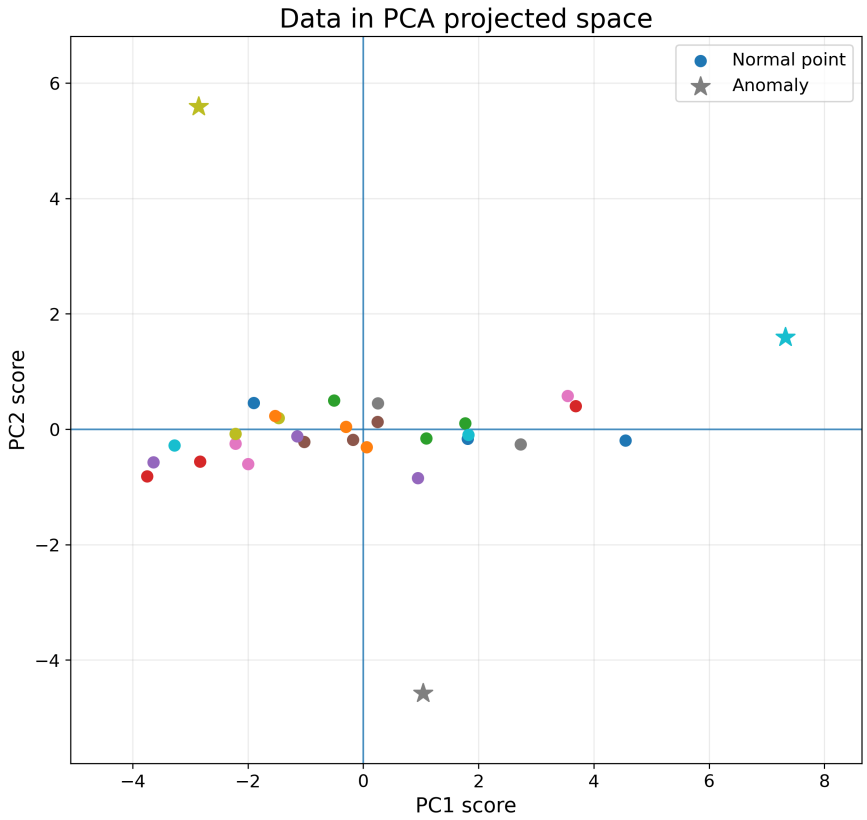


Figure 2.5: Dataset represented in the PCA coordinate system where each observation is described by its PC1 and PC2 coordinates.

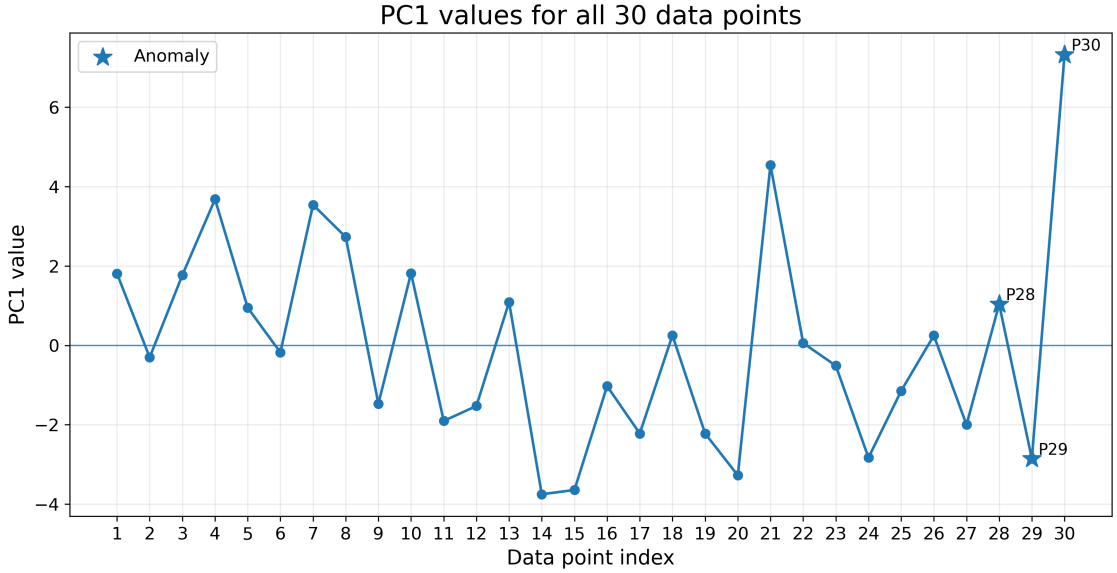


Figure 2.6: Values of the first principal component for all observations.

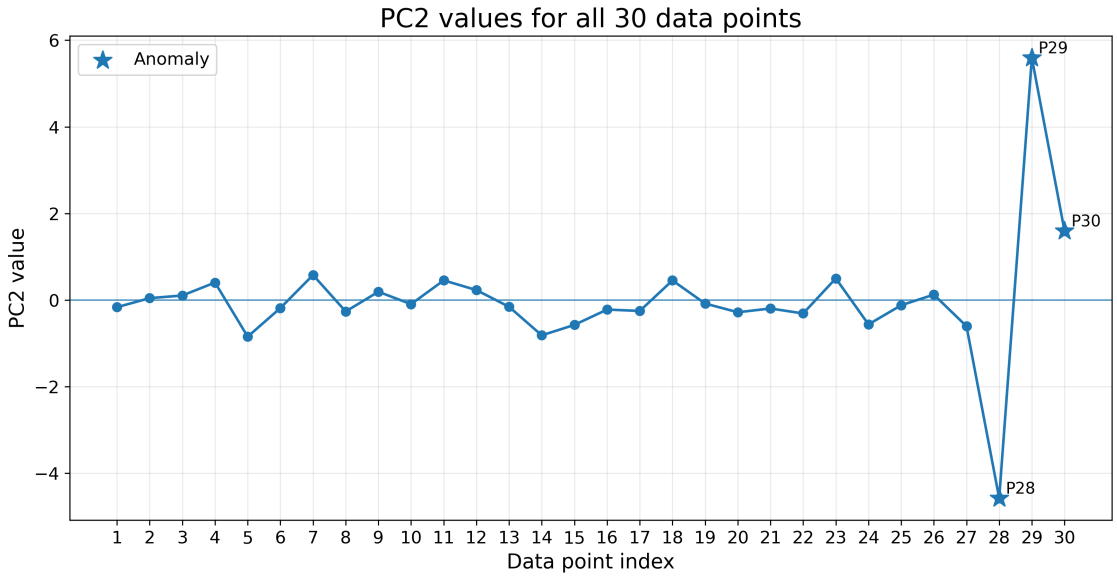


Figure 2.7: Values of the second principal component for all observations.

2.5 The CERT Dataset and Insider Cases

The CERT Insider Threat Test Dataset is a collection of synthetic datasets produced by the CERT Division of Carnegie Mellon University's Software Engineering Institute, with ExactData, LLC, under sponsorship from DARPA [5, 4]. Each dataset simulates an organization in which ordinary employees go about their work and a few simulated insiders act out a scripted malicious scenario. This thesis uses release 4.2, which simulates 1,000 employees whose recorded activity runs from 2 January 2010 to 17 May 2011. The data contain activity logs (logons and logoffs, removable-device connections, file operations, emails and web visits), organizational records (role, business unit, functional unit, department, team, and whether the employee is an IT administrator), and five personality scores for every employee.

Every log entry is an event $e = (u, \tau, s, \mathbf{a})$: a user $u \in \mathcal{U}$, a time τ, a source s (logon, device, file, email or web) and a list of attributes $\mathbf{a}$, with $|\mathcal{U}| = 1{,}000$. The unit of observation is the *user-day* $D = (u, t)$, one user on one calendar day, whose events are $E_{u,t} = \{e : e \text{ is an event of } u \text{ on day } t\}$. Every method in this thesis maps a user-day to an anomaly score $s(D) \in \mathbb{R}$, and a larger score means a more unusual day.

The dataset's answer key names each simulated insider, the scenario that insider follows, and the malicious events involved [4]. Release 4.2 contains 70 insiders in three scenarios (Table 2.2).

Table 2.2: The three insider scenarios of CERT release 4.2, following the answer key.

scenario	what the insider does	insiders
1	Starts working after hours and using a removable drive, which the user had not done before; uploads data to wikileaks.org and soon leaves.	30
2	Browses job websites and seeks work at a competitor; before leaving, copies data with a thumb drive at a much higher rate than before.	30
3	A disgruntled system administrator installs a keylogger on the supervisor's computer, logs in as the supervisor with the captured credentials and sends an alarming mass email, then leaves.	10

The language-model experiment uses the official incident window of each insider. If I_t is the calendar interval of day t, and $[a_u, b_u)$ is the answer key's incident interval, then

$$y(u,t) = \mathbb{I}[I_t \cap [a_u, b_u) \neq \varnothing] \tag{2.9}$$

for an official insider, and $y(u,t) = 0$ for other users. This identifies an *incident-window day*; it does not label every action on that day as malicious. An insider's other days are called *window-negative*, because absence from the window does not verify that all activity was benign. The calendar join uses December 28, 2009 as the day-index origin. Correct alignment gives 1,362 incident-window days from all 70 insiders among the 330,295 days with extracted sessions; another 530 calendar-window days have no session and are outside

that observation domain.

The PCA experiment uses a separate day-level feature table and its event-day flag: 966 days contain recorded malicious events (85, 861 and 20 by scenario). This is a different target from incident-window membership. Both experiments identify the same 70 official insiders, but their daily records and inputs differ, so their day-level metrics are not interchangeable.

Incident-window days account for about 0.41% of all session user-days. Calling every day negative would therefore give high accuracy while detecting no incidents. We instead evaluate rankings. A threshold on an anomaly score determines a true positive rate, the fraction of positive cases flagged, and a false positive rate, the fraction of negative cases flagged. The receiver operating characteristic (ROC) curve plots these rates as the threshold varies. Its area (AUC) is the probability that a positive case outranks a negative case, giving half credit for a tie. User-level AUC compares official insiders with non-insiders after taking each user's maximum daily score. A value of 1 is perfect ranking and 0.5 is chance-level ranking. This does not by itself measure which day an incident occurred.

2.6 PCA Experiment: Preparation and Results

The method above needs each observation as a vector of p numerical features, so the activity of each observation is summarized into one row $\mathbf{x}_i^\top$ of a matrix $X \in \mathbb{R}^{n \times p}$. Here an observation is a user-day, described by the day-level features of Le et al. [6] (Table 2.3). Release 4.2 gives $n = 330{,}452$ user-days with 502 features once identifiers and time indices are removed. Because PCA is sensitive to scale, each column is standardized with the mean and standard deviation of the training rows. Standardization needs a positive standard deviation, so the 193 columns that never vary among the training days are removed, leaving $p = 309$ features, or 299 without the organizational and personality attributes, the PCA counterpart of the behavior-only score of Chapter III.

PCA is fit on the 287,961 days of the 851 training users of Section 3.5, the users that also

Table 2.3: The day-level features of CERT release 4.2 [6], and how many of them vary among the training days. Activity features are also split by working hours and after hours and by the kind of computer used (the user's own, a shared one, another user's or the supervisor's). Files are split into six types and websites into six kinds, among them job-search, cloud-storage, leak and hacking websites. The profile attribute that does not vary is the business unit, which takes a single value in this release.

group	what is measured	features	varying
all activity	events of any kind	15	14
logons	logons	15	14
removable drives	connections and their mean duration	18	14
files	accesses by type, with mean length, depth and word count	210	132
emails	recipients, attachments, size and text length	42	33
web	visits by kind of website, with mean URL and content length	189	90
calendar	weekday or weekend	2	2
profile	organizational attributes and five personality scores	11	10
total		502	309

train the language model; all of them are benign, so PCA sees no insider and no held-out benign user. It keeps the first q principal directions U_q, with q the smallest number whose proportion of variance explained (2.5) reaches 95%; this gives $q = 71$ directions, which explain 95.1% of the variance ($q = 63$ without the profile attributes). Each observation is reduced to its scores and mapped back to the feature space with the inverse PCA transformation (2.6), and its anomaly score is its reconstruction error e_i (2.7); the mean squared error (2.8) summarizes a set of observations. Benign behavior is reconstructed well by the dominant directions of benign variation, so a large e_i marks an observation that those directions cannot explain.

The median reconstruction error is 6.6 on the days of the 79 held-out benign users and 17.0 on the 966 malicious days, and 6.4% of the malicious days exceed the 99th percentile of the training errors, against 0.5% of the held-out benign days. Scoring each user by their largest error gives the user-level AUCs of Table 2.4 and the ROC curve of Figure 2.8. For

all 70 insiders, the user AUC is 0.777. Removing profile attributes changes it little. The methods use different representations and sets of observed days; this is not a controlled comparison of model architectures. The last PCA row shows why the constant columns are removed. They include visits to leak and hacking websites and logons to the supervisor's computer, which no training day contains and which occur, in the evaluation data, almost only on malicious days. Left in their raw units, as some software does for a column with zero standard deviation, they carry 99.7% of the malicious days' squared error.

Table 2.4: PCA user-level AUC for all 70 official insiders against 79 held-out benign users, using each user's largest daily reconstruction error. Intervals resample both insider and benign users, with 10,000 draws. All rows retain 95% of training variance.

PCA input	p	q	user AUC [95% interval]
Varying features	309	71	0.777 [0.701, 0.848]
Varying behavior features	299	63	0.791 [0.716, 0.860]
All features, constants in raw units	502	71	0.832 [0.764, 0.894]

2.7 Summary

PCA describes high-dimensional observations through a small set of orthogonal directions. The discarded component gives a reconstruction error that can serve as an anomaly score. The CERT example also shows that preprocessing matters: retaining training-constant columns in raw units can make a few rare event fields dominate that error. The next chapter considers a different model of normal activity and asks which parts of its score help distinguish insiders.

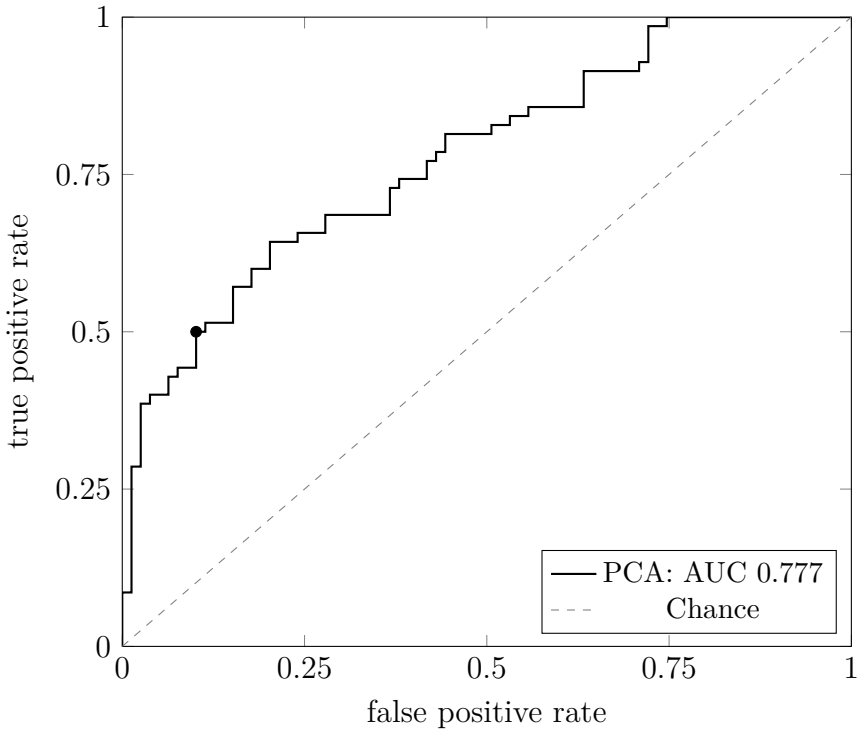


Figure 2.8: PCA ROC on all 70 insiders and 79 held-out benign users, using the 309 varying features. The marked threshold flags 35 insiders and 8 benign users. The curve is recomputed from the saved user scores of the original PCA run; no PCA refit is involved.

Chapter III

Language-Model Surprisal and Its Decomposition

3.1 From Reconstruction to Prediction

Chapter II scored an observation by the reconstruction error of a low-dimensional linear model. This chapter scores a user-day by how difficult its text record is for a language model, fine-tuned only on benign activity, to predict. Both scores ask how badly a model of normal behavior accounts for an observation [10, 11]. We then separate the prediction losses for *who* the employee is from those for *what* the employee did.

Figure 3.1 locates the computations used in this chapter and the intervention studied in Chapter IV. The model predicts successive pieces of the observed record; it does not predict whether the employee is malicious. For example, after `n_email=`, we score the probability assigned to the observed digit `3`. The equations below follow that prediction from the network's activations to the final anomaly score.

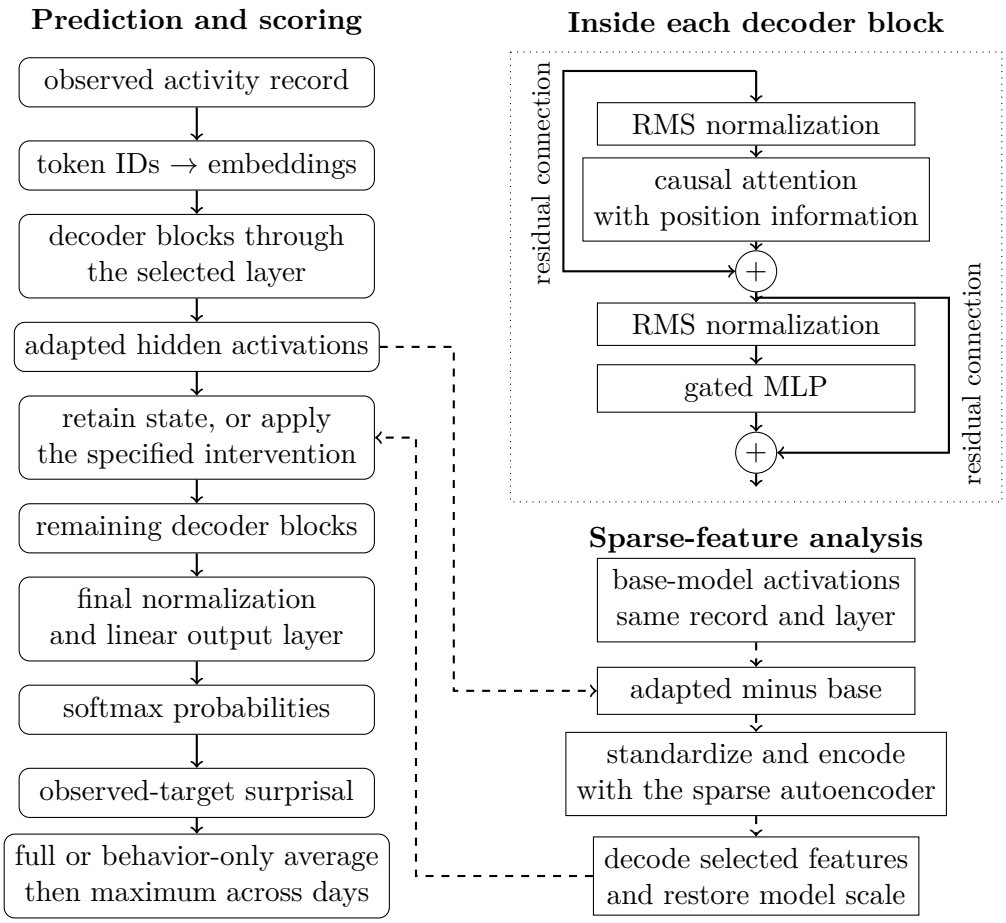


Figure 3.1: The prediction path, decoder-block structure, and sparse-feature intervention path. The 36 decoder blocks contain attention, gated MLPs, and residual additions. Solid arrows carry the scored forward pass; dashed arrows describe the analysis and edit used in Chapter IV. Ordinary scoring retains the state at the intervention site. The base activations come from a separate pass on the same input.

3.2 From Activations to Token Probabilities

An activation is a computed value. For a single unit, the calculation is

$$s = w_1 z_1 + w_2 z_2 + b, \qquad h = \phi(s), \qquad \phi(s) = \max(0, s). \tag{3.1}$$

z_1, z_2: input values

w_1, w_2: connection weights

b: bias (added offset)

s: weighted sum; h: activation

ϕ: ReLU, which clips negative inputs to zero

max: the larger argument

With fixed weights and bias, two inputs give

$$(w_1, w_2, b) = (2, -1, 1)$$

$$(z_1, z_2) = (1, 2) \implies s = 1 \implies h = 1,$$

$$(z_1, z_2) = (1, 4) \implies s = -1 \implies h = 0.$$

Thus changing an activation does not require changing a weight. For a layer with m inputs and q units, the same computation is

$$\mathbf{h} = \phi(W\mathbf{z} + \mathbf{b}), \qquad W \in \mathbb{R}^{q \times m}, \quad \mathbf{z} \in \mathbb{R}^m, \quad \mathbf{b}, \mathbf{h} \in \mathbb{R}^q. \tag{3.2}$$

Here $\mathbb{R}^m$ means a column of m real numbers; the matrix dimensions give rows by columns. Bold letters collect the scalar values above, and ϕ acts on each entry. An MLP composes such layers. The stored weights and biases form its parameters, denoted θ; its activations depend on the input. The ReLU calculation is a teaching example.

For the language model, let $x_1, \ldots, x_n$ be a record's n tokens. Position t has observed token x_t and prefix $x_{<t} = (x_1, \ldots, x_{t-1})$. The embedding map E assigns each token a vector of width d:

$$\mathbf{h}_j^{(0)} = E(x_j) \in \mathbb{R}^d, \qquad j = 1, \ldots, n.$$

The index j locates a token; the superscript k below counts completed transformer blocks. Let $H^{(k)}$ be the matrix whose row j is $(\mathbf{h}_j^{(k)})^\top$, where $\top$ means transpose. Qwen3-8B uses $L = 36$ blocks and width $d = 4096$ [15, 16]. One block computes

$$\begin{aligned} U^{(k)} &= H^{(k-1)} + \mathcal{A}_k\big(\mathcal{N}_{k,1}(H^{(k-1)})\big)\,, \\ H^{(k)} &= U^{(k)} + \mathcal{M}_k\big(\mathcal{N}_{k,2}(U^{(k)})\big)\,, \qquad k = 1, \ldots, L. \end{aligned} \tag{3.3}$$

$H^{(k)}, U^{(k)} \in \mathbb{R}^{n\times d}$	full-block and intermediate activations
$\mathcal{A}_k$	causal attention: position j reads positions at most j
$\mathcal{N}_{k,1}, \mathcal{N}_{k,2}$	row-wise RMS normalization before each branch
$\mathcal{M}_k$	gated MLP, applied separately at each position

The additions retain the incoming state and add each branch's contribution; these are the residual connections. Qwen's MLP uses a gated activation: for an input column $\mathbf{u} \in \mathbb{R}^d$,

$$\begin{aligned} \mathcal{M}_k(\mathbf{u}) &= W_{\text{down}}\left[\text{SiLU}(W_{\text{gate}}\mathbf{u}) \odot (W_{\text{up}}\mathbf{u})\right], \\ \text{SiLU}(a) &= \frac{a}{1 + e^{-a}}. \end{aligned} \tag{3.4}$$

Here a is a scalar, e is the natural exponential base, and $\odot$ multiplies corresponding entries. SiLU acts entry by entry. The gate and up matrices have dimensions $d_{\text{ff}} \times d$; the down matrix has dimensions $d \times d_{\text{ff}}$, with $d_{\text{ff}} = 12288$. Their block index is suppressed. Appendix A.6 gives the normalization and causal attention calculations. The parameters θ now collect all stored model weights, including embeddings, attention, MLPs, and output weights.

For each candidate output token v in the vocabulary $\mathcal{V}$, the output layer produces a logit:

$$o_{t,v} = \mathbf{w}_v^\top \mathcal{N}_f(\mathbf{h}_{t-1}^{(L)}). \tag{3.5}$$

Here $\mathbf{w}_v \in \mathbb{R}^d$ is the candidate's output-weight vector and $\mathcal{N}_f$ is the final normalization. The state at $t-1$ predicts token t, so it cannot use that token or later ones. Softmax converts

the logits into probabilities:

$$p_{t,v} := p_\theta(x_t = v \mid x_{<t}) = \frac{\exp(o_{t,v})}{\sum_{v' \in \mathcal{V}} \exp(o_{t,v'})}. \tag{3.6}$$

The notation := defines a quantity; | means given; $\exp(a) = e^a$. The sum includes every vocabulary member v', including v. Thus v is a candidate, while x_t is the observation whose probability we will score.

3.3 Surprisal and Benign-Only Adaptation

The observed-token loss, or *surprisal*, is

$$\ell_t = -\log p_{t,x_t}. \tag{3.7}$$

All logarithms are natural, giving losses in nats. For an illustrative vocabulary $(\mathtt{0}, \mathtt{3}, \mathtt{8})$,

$$(o_{t,v})_v = (0, \log 8, 0) \implies (p_{t,v})_v = (0.1, 0.8, 0.1) \implies \ell_t = -\log(0.8) \approx 0.223 \quad \text{if } x_t = \mathtt{3}.$$

The subscript v outside parentheses collects the candidates' entries, and $\approx$ means approximately equal. A less probable observation, with probability 0.01, has loss 4.605. The logarithm makes successive prediction losses additive:

$$-\log p_\theta(x_2, \ldots, x_n \mid x_1) = \sum_{t=2}^{n} -\log p_\theta(x_t \mid x_{<t}) = \sum_{t=2}^{n} \ell_t. \tag{3.8}$$

This is the chain rule of probability, conditioning on the first token. It does not assume token independence. The sum is the negative log-likelihood of the remaining observed sequence.

We adapt pretrained Qwen3-8B on benign records using LoRA corrections:

$$W' = W + GH, \qquad G \in \mathbb{R}^{d_{\text{out}} \times r}, \quad H \in \mathbb{R}^{r \times d_{\text{in}}}, \qquad r = 16. \tag{3.9}$$

Here W is a selected frozen matrix with d_{in} inputs and d_{out} outputs; W' is its adapted value and r limits the correction's rank. Only the factors G, H are trained. The fixed LoRA scale is absorbed into G, and the equation describes evaluation with training dropout (random masking of adapter inputs) disabled. The correction's output lies in the span of G's r columns. This rank bound applies to each matrix, not the full nonlinear network.

Let θ_0 denote the frozen base parameters and η collect the trainable factors, so $\theta = (\theta_0, \eta)$. Adaptation minimizes

$$J(\theta_0, \eta) = \sum_{D \in \mathcal{B}} \sum_{t=2}^{n_D} -\log p_{\theta_0,\eta}\big(x_t^{(D)} \mid x_{<t}^{(D)}\big). \tag{3.10}$$

The set $\mathcal{B}$ contains benign training days; D identifies a record, n_D its token count, and (D) marks its tokens. J sums their prediction losses. QLoRA stores the frozen base weights in 4-bit form to reduce memory use [13, 14]. The factor count is $r(d_{\text{in}} + d_{\text{out}})$ rather than $d_{\text{in}} d_{\text{out}}$: 131,072 instead of 16,777,216 for a 4,096-by-4,096 matrix, or about 0.78%.

Benign-only refers to this adaptation, not Qwen's original pretraining. After adaptation, both θ_0 and η stay fixed. Evaluation measures loss without updating either.

3.4 Day Scores and the Profile–Behavior Decomposition

For one day D, define its scored positions and average loss by

$$\mathcal{T} = \{2, \ldots, n\}, \qquad N = |\mathcal{T}| = n - 1, \qquad \bar{\ell}(D) = \frac{1}{N} \sum_{t \in \mathcal{T}} \ell_t. \tag{3.11}$$

Here $n \geq 2$ is the record length, $|\mathcal{T}|$ counts its scored positions, and the bar denotes an average. The first token is unscored. For employee u with observed-day set $\mathcal{D}_u$, the user

score is

$$\bar{\ell}(u) = \max_{D \in \mathcal{D}_u} \bar{\ell}(D). \tag{3.12}$$

This selects their most unusual day. Longer histories provide more opportunities for a high maximum; a high score alone does not establish malice.

Partition the scored positions into profile (P) and behavior (B):

$$\mathcal{T} = \mathcal{T}_P \cup \mathcal{T}_B, \qquad \mathcal{T}_P \cap \mathcal{T}_B = \varnothing.$$

The union $\cup$ includes both groups; the intersection $\cap$ is empty ($\varnothing$), so no position belongs to both. For class $c \in \{P, B\}$,

$$N_c = |\mathcal{T}_c|, \qquad L_c = \sum_{t \in \mathcal{T}_c} \ell_t, \qquad \bar{\ell}_c = \frac{L_c}{N_c}.$$

These are its count, total loss, and mean loss. Both classes are nonempty. Suppressing the day argument, put $N = N_P + N_B$ and $L = L_P + L_B$. Then

$$\bar{\ell} = \frac{L}{N} = \frac{N_P}{N}\bar{\ell}_P + \frac{N_B}{N}\bar{\ell}_B, \tag{3.13}$$

and removing profile predictions from the score gives

$$\bar{\ell}_B = \frac{L_B}{N_B} = \frac{L - L_P}{N - N_P}. \tag{3.14}$$

Both the loss and token count are removed. The input is unchanged: behavior predictions can still depend on preceding profile text. For each score view, user maxima are recalculated using that daily score.

For an illustration, let both days have one profile and four behavior targets, giving weights 0.2 and 0.8:

Table 3.1: Class means and scores for two illustrative days. The full score weights profile loss by 0.2 and behavior loss by 0.8.

day	$\bar{\ell}_P$	$\bar{\ell}_B$	full $\bar{\ell}$	behavior only
A (malicious)	0.1054	0.9324	0.7670	0.9324
B (benign)	2.9957	0.4125	0.9291	0.4125

The profile contribution reverses the ordering under the full score; behavior-only scoring restores it. These are illustrative values. Appendix A.4 shows why subtracting class means does not produce the behavior-only score.

3.5 Experimental Design

We use CERT r4.2 sessions, extracted from logon to logoff on one computer [6]. Each user's daily sessions are written as activity measurements in text, rather than email message bodies:

line	content	scored class
`DAY`, `PSY`	organization and personality	profile (P)
`SESSIONS`, `SES`	session count and activity	behavior (B)

Class follows the target token's text location; residual text belongs to behavior. The profile group includes the week field, so it is not purely an identity category. Appendix A.7 contains the full record, tokenizer example, and representation caveats.

There are 470,611 sessions on 330,295 observed user-days. Days without extracted sessions are outside this experiment. All days of a user stay in the same group. Let $\mathcal{U}_{\text{train}}$, $\mathcal{U}_{\text{val}}$, and $\mathcal{U}_{\text{ins}}$ denote training users, held-out benign users, and official insiders: All insiders enter the last group. A fixed hash of each other user's identifier assigns about one tenth to the held-out group. Adaptation makes one pass over the 851 training users' 287,827 days.

Table 3.2: User-disjoint splits of CERT release 4.2: no user belongs to more than one group.

group	users	user-days	incident-window days
$\mathcal{U}_{\text{train}}$ (training)	851	287,827	0
$\mathcal{U}_{\text{val}}$ (held-out benign)	79	27,026	0
$\mathcal{U}_{\text{ins}}$ (insiders)	70	15,442	1,362

Held-out means excluded from adaptation; evaluation users were examined during research development and are not an untouched test set.

The evaluation pool has 42,468 days from 149 users, including 1,362 incident-window days under the calendar rule in Section 2.5. An insider's other days are window-negative, not verified benign. The correction changed evaluation labels and score coverage, not the model or its benign training membership.

Each of 70 evaluation folds compares one insider with the same 79 benign users. Day labels distinguish incident-window days from the remaining days in that fold; user labels distinguish insiders from benign employees. The model stays fixed across folds. The reported metrics are equal-insider fold means:

- *ROC-AUC*: the share of positive–negative pairs correctly ordered, with half credit for ties. One is perfect ranking; 0.5 is chance-level ranking. Its threshold curve plots the fraction of positives flagged against the fraction of negatives flagged.

- *Average precision (AP)*: precision weighted by increases in recall as the score threshold is lowered. Precision is the positive fraction of flagged cases; recall is the fraction of positives found.

Mean fold day prevalence is 0.0714%, versus 3.21% when all days are pooled. AP must be interpreted using its fold populations. Appendix A.5 gives the user-AUC identity; it does not extend to the day metrics.

3.6 Results and the Connection to Interventions

Table 3.3: Corrected CERT r4.2 score decomposition: 42,468 days, including 1,362 incident-window days, from 70 insiders and 79 held-out benign users. All views retain the same input and model. AUC and AP columns are equal-insider fold means.

Scored targets	day AUC	day AP	user AUC
All tokens	0.540	0.004859	0.589
Behavior tokens	0.820	0.025238	0.825
Profile tokens	0.482	0.001353	0.476

Behavior-only scoring raises user AUC from 0.589 to 0.825: insiders outrank benign users in about 82.5% rather than 58.9% of pairs, counting ties by halves. This is a ranking result, not classification accuracy at an alert threshold.

The paired gain is 0.236, with descriptive 95% user-bootstrap interval $[0.185, 0.289]$. The bootstrap resamples the 70 insiders with replacement, pairing their two AUCs, while keeping the model and benign cohort fixed. The profile-only estimate slightly below chance does not establish a reliable reversed ranking.

Changing the scored targets leaves all hidden activations unchanged. An intervention instead edits a chosen layer and recomputes the downstream predictions. Using the activation matrix $H^{(k)}$ from (3.3), write

$$\widetilde{H}^{(k)} = \mathcal{I}(H^{(k)}), \qquad \Delta\bar{\ell}_B = \mathcal{S}_{B,k}(\widetilde{H}^{(k)}) - \mathcal{S}_{B,k}(H^{(k)}). \tag{3.15}$$

Here $\mathcal{I}$ is the edit, the tilde marks the edited state, and $\mathcal{S}_{B,k}$ runs the remaining blocks, output layer, and behavior-loss calculation with fixed parameters and observed targets. Thus $\Delta\bar{\ell}_B$ measures a change in prediction loss caused by the specified internal edit.

Chapter IV specifies this edit through selected sparse features of the adapted-minus-base activations. Its corrected experiment removes their decoded contribution. Steering and patching are other intervention policies; Appendix A.8 defines them and the stronger

requirement of causal abstraction [21]. The current results do not establish a formal causal abstraction.

The score result concerns one model, representation, and synthetic benchmark. It does not establish transfer to real organizations, identify which internal computation uses profile information, or show that a user's highest-scoring day is their incident day.

Chapter IV

Interpretable Dimensionality Reduction of the Fine-Tuned Model

4.1 Overview and Motivation

Chapter III examined the score from the outside: which tokens its surprisal comes from. *Mechanistic interpretability* asks what the model computes internally [17, 18]. This chapter answers that question with a form of dimensionality reduction designed for interpretation. PCA (Chapter II) reduces data to a few dense directions, each of which mixes many original features. A *sparse autoencoder* instead describes each observation with a few directions drawn from a much larger dictionary; because only a few are active at once, the directions, or *features*, provide candidate patterns to investigate. Sparsity alone does not guarantee that a feature has one interpretable meaning. We apply it to the change that benign fine-tuning made to the model's internal representations, and test two things about the features that best separate malicious from benign days: where they activate, and whether editing them changes the anomaly score.

4.2 Hidden States and the Adaptation Difference

The model processes text through a stack of 36 layers. After each layer, each token t is represented by a *hidden state* $\mathbf{h}_t \in \mathbb{R}^d$ with $d = 4096$. Running the same text through the

model before fine-tuning (the *base* model) and after it (the *adapted* model) gives two hidden states for every token. Their difference,

$$\boldsymbol{\delta}_t \;=\; \mathbf{h}_t^{\text{ada}} - \mathbf{h}_t^{\text{base}} \;\in\; \mathbb{R}^d, \tag{4.1}$$

is the net difference in token t's representation between the two model runs. We study the hidden states after layer 26.

4.3 A Sparse Dictionary and a Worked Example

Recall the PCA reconstruction of Chapter II: a centered observation is projected onto $q < p$ orthonormal directions and reconstructed by their weighted sum. A sparse autoencoder (SAE) also minimizes reconstruction error, but uses an overcomplete dictionary of $m > d$ directions and activates at most k directions for each token [20].

Let $\boldsymbol{\mu}$ and the positive coordinate scales $\boldsymbol{\sigma}$ be estimated from the dictionary's fitting data, and put $S = \operatorname{diag}(\boldsymbol{\sigma})$. The implementation first standardizes each difference vector and then encodes it:

$$\begin{aligned} \widetilde{\boldsymbol{\delta}}_t &= S^{-1}(\boldsymbol{\delta}_t - \boldsymbol{\mu}), \\ \mathbf{z}_t &= \operatorname{TopK}_k\Big(\operatorname{ReLU}(W_{\text{enc}}\widetilde{\boldsymbol{\delta}}_t + \mathbf{b}_{\text{enc}})\Big). \end{aligned} \tag{4.2}$$

ReLU sets negative encoder outputs to zero, and TopK retains the k largest entries. Some of those entries may already be zero, so the code has at most k nonzero values. The reconstruction in the model's original coordinates is

$$\widehat{\boldsymbol{\delta}}_t = \boldsymbol{\mu} + SW_{\text{dec}}\mathbf{z}_t = \boldsymbol{\mu} + S\sum_f z_{f,t}\mathbf{w}_f, \tag{4.3}$$

where $\mathbf{w}_f$ is decoder column f. Training minimizes squared reconstruction error in standardized coordinates. The decoder has no additional learned bias.

Here $d = 4096$, $m = 8192$ and $k = 4$. The corrected dictionary is fitted for 20 epochs on two million sampled token differences at layer 26. The reconstruction objective has no label term, but the sampling procedure is label-aware: it preserves the 171,917 token rows from discovery-positive days and samples the remaining rows from the rest of the evaluation population. Confirmation-user inputs can therefore enter dictionary fitting, although their labels do not prioritize that sampling or enter feature ranking. This is a *transductive* dictionary, not a dictionary fitted on an independent training population. The low-dimensional object is each token's sparse code; the dictionary as a whole has more directions than the original space.

For an illustration, take $d = 2$, a dictionary of $m = 3$ unit directions $\mathbf{w}_1 = (1,\ 0)$, $\mathbf{w}_2 = (0,\ 1)$ and $\mathbf{w}_3 = (0.6,\ 0.8)$, one active feature ($k = 1$), $\boldsymbol{\mu} = \mathbf{b}_{\text{enc}} = \mathbf{0}$, $S = I$, and $W_{\text{enc}} = W_{\text{dec}}^{\top}$. For $\boldsymbol{\delta} = (1.2,\ 1.6)$ the scores are $\mathbf{a} = (1.2,\ 1.6,\ 2.0)$, so feature 3 is kept and $\widehat{\boldsymbol{\delta}} = 2.0\,\mathbf{w}_3 = (1.2,\ 1.6)$ with zero error. With only the two axes available, the best single direction would leave an error of $1.2^2 = 1.44$. A different token, $\boldsymbol{\delta}' = (1.0,\ 0.1)$, is described by feature 1 with error 0.01: each token uses the few features that fit it best, which is *sparse coding*.

4.4 Choosing the Features to Test

The 70 insiders are divided into 35 discovery and 35 confirmation users. Feature selection uses corrected discovery labels and excludes the confirmation users. For each feature f, the ranking statistic is

$$g_f = \bar{z}_f^{(+)} - \bar{z}_f^{(-)}, \tag{4.4}$$

the difference between its mean token activation in the positive and negative discovery populations. The selected set $\mathcal{F}$ contains the five largest gaps. The active control set $\mathcal{C}$ uses five other features with gaps near zero and nonzero activation on at least 0.2% of ranking tokens. This activity threshold does not match either the activation frequency or the physical size

of the selected edit.

The corrected selected IDs are 3843, 3230, 3199, 3975 and 1394. IDs identify columns of this particular fitted dictionary; they are not named concepts and cannot be compared directly with IDs from another fit. The confirmation group excludes users from feature ranking, but those users were inspected earlier in the research and enter unlabelled dictionary fitting. It is therefore a check across ranking groups, not an independent confirmation set.

4.5 Where Features Activate

For a feature f and a token class c, the share of the feature's total activation that lands on class-c tokens is

$$\rho_f(c) \;=\; \frac{\sum_{t\in\mathcal{T}_c} z_{f,t}}{\sum_{t\in\mathcal{T}} z_{f,t}}, \tag{4.5}$$

and its *enrichment* compares that share with the class's share of the tokens,

$$E_f(c) \;=\; \frac{\rho_f(c)}{N_c/N}.$$

When the denominator is positive, an enrichment of 1 means activation mass is proportional to token frequency; it does not measure how often the feature fires. Larger values mean more activation mass per token on class c, up to the maximum $1/(N_c/N)$ reached when all of a feature's activation lands on that class.

If a feature has zero total activation in a group, its location fraction is undefined. Table 4.1 reports session-token mass and the number of confirmation users on whose incident-window days the feature activates at least once. Four features place virtually all of their incident-window activation mass on session tokens. Feature 3843 has a mixed location and much weaker confirmation activity. Location answers where a feature activates. It does not identify all information carried there: a session state can depend on the profile text preceding it. Nor does location alone show which predictions would change if that feature were edited.

Table 4.1: Corrected r4.2 attribution. The discovery group has 695 incident-window days from 35 users; confirmation has 667 days from 35 users. Percentages pool activation mass within each group. Values displayed as 100.00% are rounded.

Feature	discovery SES (%)	confirmation SES (%)	active users
3843	91.45	84.95	13/35
3230	100.00	100.00	32/35
3199	100.00	100.00	24/35
3975	100.00	100.00	30/35
1394	100.00	100.00	30/35

4.6 Causal Test: Removing Features and Re-Scoring

Attribution is descriptive. To test whether the selected representation contributes to prediction, remove its decoded contribution from the adapted hidden state:

$$\mathbf{h}'_t = \mathbf{h}^{\text{ada}}_t - \alpha S \sum_{f \in \mathcal{F}} z_{f,t} \mathbf{w}_f, \qquad 0 \le \alpha \le 1. \tag{4.6}$$

This leaves the original hidden state and its reconstruction residual intact except for the specified subtraction. At $\alpha = 0$ it is an exact no-op; at $\alpha = 1$ it removes the selected decoded contribution. Tokens with no selected activation remain unchanged. Re-encoding the edited state need not reproduce the intended sparse code, so the experiment is defined by this physical subtraction, not by assuming independent feature coordinates.

The remaining model layers predict the same target tokens. For a day D, let $\Delta_{\mathcal{F}}(D) = \bar{\ell}'_B(D) - \bar{\ell}_B(D)$ be the change in behavior-target loss. A positive value means prediction became worse. Two controls answer different questions: removal of the active-control features tests a separate feature-selection policy, while a random dictionary direction at the same edited positions is matched to the realized per-token edit norm within 5%. Matching is checked after numerical rounding in the model's precision. The fixed feasibility policy for those dictionary controls is retained from the corrected recovery protocol; it is not a new, independently preregistered test. The active-feature control is not dose matched.

The primary removal strength is fixed at $\alpha = 1$. For each incident-window receiver, the experiment also uses a same-user window-negative day, chosen by nearest calendar day, preferring the same weekday and using fixed hash tie-breaking. Negative days can be reused. For a receiver pair (D^+, D^-), the relative window effect is

$$\begin{aligned}\tau(D^+, D^-) = [\Delta_{\mathcal{F}}(D^+) - \Delta_{\mathcal{C}}(D^+)] \\ - [\Delta_{\mathcal{F}}(D^-) - \Delta_{\mathcal{C}}(D^-)].\end{aligned} \tag{4.7}$$

A positive value would indicate a larger selected-versus-active response inside the incident window. Pairing is observational: even a positive value would not by itself prove an attack-specific circuit. Days are averaged within each user, then users are weighted equally; zero edits and repeated negative-day appearances retain their prescribed weights.

4.7 Corrected Mechanistic Results

The corrected run covers all 1,362 incident-window days and 380 unique matched window-negative days, giving 1,742 receivers and 13,936 scored rows across eight conditions. The primary summary uses the 667 incident-window days of the 35 confirmation users. Zero-edit and implementation checks passed, and the aggregate estimates were independently reproduced.

Table 4.2: Behavior-target loss changes at full removal strength on the 35 confirmation users, in nats per token. Intervals are descriptive, unadjusted 95% intervals from 10,000 user-bootstrap draws. The final row is the incident-minus-window-negative contrast in (4.7); the other rows describe incident-window receivers.

Contrast	mean	95% interval
Selected minus baseline	0.004411	[0.001812, 0.007283]
Selected minus matched direction	0.005531	[0.002886, 0.008625]
Selected minus active control	0.004275	[0.001709, 0.007179]
Incident minus window-negative	−0.004599	[−0.009430, −0.000616]

Removing the selected contribution raises behavior loss relative to baseline and to both

specified controls. This supports a contribution to prediction under this edit. However, the relative response is smaller on incident-window days than on their matched window-negative days: the final contrast is negative. The results therefore do not support an enhanced attack-specific response under this policy.

An intervention's effect on prediction loss is also different from an improvement in anomaly detection. These estimates do not demonstrate better ranking, semantic identification of the features, or mediation of the profile's influence. Together with Chapter III, they support two bounded conclusions: score composition affects insider ranking, and the selected sparse directions contribute to behavior prediction. The second does not explain the first by itself.

4.8 Limitations and Conclusions

The analysis uses one adapted model and one synthetic benchmark release. Model-seed variation and transfer to real organizational data remain untested here. The layer and dictionary settings were informed by earlier experiments; the corrected dictionary is transductive, its sample prioritizes discovery-positive rows, and its confirmation users were previously inspected. Feature ranking excludes those users, but that separation does not make the entire analysis independent.

The controls have distinct scopes. Active-feature removal compares selection policies with different natural edit sizes. Dictionary directions match edited positions and realized norms, but do not enumerate every possible alternative intervention. Same-user negative days control some user differences but are not verified benign ground truth. The reported intervals are conditional on the fixed model, dictionary, feature sets, pairs and controls.

Finally, an activation at a behavior-token position need not represent behavior alone. Interpretable labels for individual features require further evidence, and these joint edits do not identify a complete computational circuit. The corrected ablation is the causal test

reported in this thesis; the earlier donor-search result is not carried forward as current evidence after label correction.

Appendix A

Supporting Mathematics for Language-Model Scoring

This appendix supplies the derivations supporting Chapter III. The main chapter develops the prediction task and its scoring rule; the arguments below explain why the logarithm is useful, how it relates to the output scores, and how the evaluation averages are constructed. All logarithms here are natural logarithms.

A.1 Why Additive Surprise Has a Logarithmic Form

Let p be an event's probability, with $0 < p \leq 1$, and let $S(p)$ be the amount of surprise assigned to it. Require that S strictly decreases as p increases: rarer events are more surprising. Let q be the probability of a second, independent event, with $0 < q \leq 1$. Independence makes the probability of both events pq. Requiring their surprises to add gives

$$S(pq) = S(p) + S(q). \tag{A.1}$$

This is a requirement on the surprise measure for independent events; it does not assert that successive text tokens are independent.

To find its form, introduce nonnegative real numbers $x = -\ln p$ and $y = -\ln q$, so $p = e^{-x}$ and $q = e^{-y}$. Here e is the base of the natural exponential. Define a new function $g(x) = S(e^{-x})$. Equation (A.1) becomes

$$g(x + y) = g(x) + g(y).$$

The function g is increasing because e^{-x} decreases with x and S decreases with probability. This additive relation is Cauchy's functional equation. Its increasing solutions on nonnegative real numbers have the form $g(x) = cx$, where c is a positive constant. Substitution therefore gives

$$S(p) = -c \ln p, \qquad c > 0. \tag{A.2}$$

The constant c sets the measurement unit. Choosing $c = 1$ gives surprisal in nats, as used in Chapter III [19]. A certain event, with $p = 1$, has zero surprise. As p approaches zero, surprise grows without bound. Every tenfold decrease in probability adds $\ln 10$ nats. Applying the logarithm to the chain of conditional token probabilities gives (3.8) without requiring independent tokens.

A.2 Surprisal Written in Terms of Logits

Fix one prediction position t, where x_t is the observed token and $x_{<t}$ is the preceding sequence. Let θ denote the fixed model parameters and $\mathcal{V}$ its vocabulary. Its size is $V = |\mathcal{V}|$, where the bars count members of the set. For each candidate token v in the vocabulary, let o_v be its output logit, and let $\mathbf{o}$ collect all those logits. Define the *log-sum-exp* function, abbreviated LSE, by taking the logarithm of the sum of exponentiated logits. Substituting softmax into the observed token's surprisal ℓ_t gives

$$\begin{aligned} \ell_t &= -\log p_\theta(x_t \mid x_{<t}) = \mathrm{LSE}(\mathbf{o}) - o_{x_t}, \\ \mathrm{LSE}(\mathbf{o}) &= \log \sum_{v \in \mathcal{V}} e^{o_v}. \end{aligned} \tag{A.3}$$

The subscript x_t selects the logit of the observed token. The summation adds over all vocabulary members, and e is the natural exponential base.

Let $o_{\max} = \max_{v \in \mathcal{V}} o_v$ be the largest logit. The sum of exponentials includes at least one

term $e^{o_{\max}}$, and none of its V terms is larger than that term. Thus

$$e^{o_{\max}} \leq \sum_{v \in \mathcal{V}} e^{o_v} \leq V e^{o_{\max}}.$$

Taking logarithms, which preserve these inequalities, gives

$$o_{\max} \leq \mathrm{LSE}(\mathbf{o}) \leq o_{\max} + \log V.$$

For two candidate tokens a and b, the common softmax denominator cancels in their probability ratio:

$$\frac{p_\theta(x_t = a \mid x_{<t})}{p_\theta(x_t = b \mid x_{<t})} = e^{o_a - o_b}.$$

Only logit differences determine the probabilities. Having the largest logit alone does not guarantee a small surprisal: many other candidates may have similar logits and share the probability.

A.3 Maximum Likelihood in a Frequency Model

Consider a model that assigns a fixed probability to each of K token types, regardless of preceding text. Let j index those types from 1 to K, let N_j count occurrences of type j in the training observations, and let $N = \sum_{j=1}^{K} N_j$ be the total count. Write p_j for the probability assigned to type j and $\mathbf{p} = (p_1, \ldots, p_K)$ for their collection. For the interior calculation, suppose each N_j is positive and each p_j is positive.

The summed negative log-likelihood is $-\sum_{j=1}^{K} N_j \log p_j$, subject to the probabilities adding to one. Introduce a real number λ, the Lagrange multiplier, which enforces this constraint. Define the Lagrangian $\mathcal{L}(\mathbf{p}, \lambda)$ to be the objective plus λ times the constraint residual:

$$\mathcal{L}(\mathbf{p}, \lambda) = -\sum_{j=1}^{K} N_j \log p_j + \lambda \left(\sum_{j=1}^{K} p_j - 1 \right).$$

The parenthesized expression is zero exactly when the probabilities sum to one. Differentiating with respect to each p_j while holding the other quantities fixed, and setting the derivative to zero, gives

$$\frac{\partial \mathcal{L}}{\partial p_j} = -\frac{N_j}{p_j} + \lambda = 0, \qquad p_j = \frac{N_j}{\lambda}.$$

The symbol ∂ denotes this partial derivative. Summing the last equality over token types and imposing a total probability of one gives $\lambda = N$. Hence the minimizing probabilities are

$$\widehat{p}_j = \frac{N_j}{N}, \qquad j = 1, \ldots, K. \tag{A.4}$$

The hat marks a fitted value. The objective is convex: along a straight segment between two positive probability vectors, its value lies at or below the line joining the endpoint values. Thus this stationary point gives the minimum under the constraint. If additional types have zero observed count, the optimum in the unrestricted frequency model assigns them zero probability. This frequency calculation illustrates the objective; a contextual language model with shared parameters and finite softmax logits is a different model and need not reproduce these exact frequencies.

A.4 Why Subtracting Class Means Gives a Different Score

For one day, let N_P and N_B count scored profile and behavior tokens, and let $N = N_P + N_B$ be their total. Assume both classes are nonempty. Let $\bar{\ell}_P$ and $\bar{\ell}_B$ be their average surprisals, and let $\bar{\ell}$ be the average across all scored tokens. From (3.13),

$$\begin{aligned} \bar{\ell} - \bar{\ell}_P &= \frac{N_P}{N}\bar{\ell}_P + \frac{N_B}{N}\bar{\ell}_B - \bar{\ell}_P \\ &= \frac{N_B}{N}\big(\bar{\ell}_B - \bar{\ell}_P\big). \end{aligned} \tag{A.5}$$

The result is a scaled difference between class means, not the behavior mean. For a numerical illustration, take two days, named C and D, each with behavior-token share $N_B/N = 0.8$. Give Day C profile mean 3.0 and behavior mean 1.2, and Day D profile mean 0.2 and behavior mean 0.8. Subtracting means gives $0.8(1.2 - 3.0) = -1.44$ for C and $0.8(0.8 - 0.2) = 0.48$ for D. It ranks D above C even though C's behavior has higher surprisal. Removing a token class from an average requires removing both its summed loss and its token count, as in (3.14).

A.5 User AUC and Equal-Insider Fold Averages

Fix one scoring rule. Let M be the number of insider users and n_- the number of benign comparison users, both positive. The minus subscript labels the negative class; it does not mean subtraction. Let m index insiders from 1 to M and j index benign users from 1 to n_-. Write s_m^+ for insider m's maximum daily score and s_j^- for benign user j's maximum daily score. The superscripts + and − label positive and negative classes, not powers.

Define the indicator $\mathbb{I}[\text{condition}]$ to be 1 when the enclosed condition is true and 0 otherwise. The user AUC for the fold containing insider m is

$$\text{AUC}_m = \frac{1}{n_-} \sum_{j=1}^{n_-} \left(\mathbb{I}[s_m^+ > s_j^-] + \tfrac{1}{2} \mathbb{I}[s_m^+ = s_j^-] \right). \tag{A.6}$$

Each comparison contributes 1 if the insider outranks the benign user, one half for a tie, and 0 otherwise. Let a_{mj} name that contribution for pair (m, j). The overall equal-insider mean, denoted AUC, therefore satisfies

$$\text{AUC} = \frac{1}{M} \sum_{m=1}^{M} \text{AUC}_m = \frac{1}{M} \sum_{m=1}^{M} \frac{1}{n_-} \sum_{j=1}^{n_-} a_{mj} = \frac{1}{Mn_-} \sum_{m=1}^{M} \sum_{j=1}^{n_-} a_{mj}.$$

Every insider is compared with the same benign users, so this is also the pooled user AUC over all Mn_- pairs. That identity does not establish an analogous equality for day-level fold

averages or AP, whose comparison populations and weights differ.

For illustration, let four benign users have full scores $(0.93, 0.70, 0.85, 0.60)$ and one insider have score 0.77. The insider outranks two benign users, giving AUC $2/4 = 0.50$. If the behavior-only scores of those users are $(0.41, 0.55, 0.70, 0.48)$ and the insider's is 0.93, the insider outranks all four, giving AUC $4/4 = 1.00$. These examples explain the calculation and are not reported experimental observations.

A.6 Normalization and Causal Attention

For a vector $\mathbf{z} \in \mathbb{R}^m$, RMS normalization is

$$\mathcal{N}(\mathbf{z}) = \frac{\boldsymbol{\gamma} \odot \mathbf{z}}{\sqrt{\epsilon + \frac{1}{m}\sum_{i=1}^{m} z_i^2}}. \tag{A.7}$$

Here m is the vector length, i its coordinate index, $\boldsymbol{\gamma} \in \mathbb{R}^m$ a learned scale, $\epsilon > 0$ a fixed stabilizing constant, and $\odot$ entrywise multiplication. Each normalization in (3.3) has its own scale parameters and operates on each token row separately.

For one attention head, let $\mathbf{q}_j$, $\mathbf{k}_s$, and $\mathbf{v}_s$ be its query at position j, key at position s, and value at position s, each with d_h coordinates. The attention weights a_{js} and the head output $\mathbf{y}_j$ are

$$\begin{aligned} a_{js} &= \frac{\exp(\mathbf{q}_j^\top \mathbf{k}_s/\sqrt{d_h})}{\sum_{u=1}^{j}\exp(\mathbf{q}_j^\top \mathbf{k}_u/\sqrt{d_h})}, && 1 \le s \le j, \\ a_{js} &= 0, && s > j, \\ \mathbf{y}_j &= \sum_{s=1}^{j} a_{js}\mathbf{v}_s. \end{aligned} \tag{A.8}$$

The index u runs over permitted key positions in the denominator; $\top$ is transpose, and $\sqrt{d_h}$ scales the dot products. The zero weights prevent position j from reading future positions. Concatenating the head outputs and applying a learned output matrix gives the attention branch contribution $\mathcal{A}_k$.

In Qwen, queries and keys in this formula are the projected vectors after query/key normalization and position-dependent rotary transformations. Multiple query heads share key/value projections through grouped-query attention [15]. These operations are included in $\mathcal{A}_k$; the weighted sum is its causal mixing step. The MLP then transforms each position separately, while residual addition preserves a path for the incoming activation.

A.7 Complete Activity Record and Representation Details

This is the benign CERT r4.2 record for user `AAF0535`, day 7. The session line is wrapped only for display.

```
DAY week=1 project=na role=39 b_unit=0 f_unit=4 dept=15 team=11 itadmin=0
PSY O=17 C=21 E=36 A=33 N=31
SESSIONS total=1 kept=1
SES idx=0 pc=0 isworkhour=1 isafterhour=0 isweekend=0 isweekendafterhour=0
    duration=489 n_concurrent_sessions=1 start_with=1 end_with=1
    ses_start=8.933 ses_end=17.083 n_allact=34 n_logon=1 n_usb=0
    usb_mean_usb_dur=0 n_file=0 file_n-disk1=0 file_n_exef=0 n_email=3
    email_mean_n_atts=1.333 n_http=29 http_n_jobf=0 http_n_cloudf=0
    http_n_leakf=0 http_n_hackf=0
```

The employee had one working-hours session on their usual computer, lasting 489 minutes, with 3 emails and 29 web requests. The letters `O`, `C`, `E`, `A`, and `N` denote openness, conscientiousness, extraversion, agreeableness, and neuroticism.

For the stored representation and tokenizer,

$$\texttt{dept=15} \longrightarrow (\texttt{dept}, \texttt{=}, 1, 5).$$

The arrow denotes tokenization. This complete record contains $N = 212$ scored targets:

$N_P = 54$ profile and $N_B = 158$ behavior targets, using the counts defined in Section 3.4. Class follows location, so the same digit can belong to either group. Each scored prediction uses the observed preceding tokens.

Days contain one to nine sessions, with two thirds containing one. The longest records have about 1,300 tokens, within the model's input limit. This is within-day likelihood scoring, not next-day forecasting or generation of replacement activity.

The historical conversion program omitted four hyphenated fields: `file_n-to_usb1`, `file_n-from_usb1`, `file_n-file_act3`, and `file_n-disk1`. If a missing field appears as zero, that is a conversion default, not a measured absence of activity. The findings apply to this recorded representation.

A.8 Intervention Policies and Causal Abstraction

The same downstream computation can be tested with different edits. At a fixed layer and token position, let $\mathbf{h} \in \mathbb{R}^d$ be the original activation and $\mathbf{h}'$ its replacement. Common choices are

$$\begin{aligned} &\text{direction steering:} && \mathbf{h}' = \mathbf{h} + \alpha\mathbf{v}, \\ &\text{coordinate patching:} && \mathbf{h}'_J = \mathbf{h}^{\text{src}}_J, \quad \mathbf{h}'_{\bar{J}} = \mathbf{h}_{\bar{J}}, \\ &\text{feature removal:} && \mathbf{h}' = \mathbf{h} - \alpha\mathbf{c}_{\mathcal{F}}. \end{aligned} \tag{A.9}$$

d	activation width
α	chosen scalar intervention strength
$\mathbf{v} \in \mathbb{R}^d$	chosen steering direction
$J, \bar{J}$	selected coordinates and their complement
$\mathbf{h}^{\text{src}}$	donor state at the same layer and an aligned position
$\mathbf{c}_{\mathcal{F}} \in \mathbb{R}^d$	decoded contribution of selected feature set $\mathcal{F}$

The corrected experiment uses the final policy, with the physical contribution defined by (4.6). A joint change in prediction loss supports a role under that edit; it does not by itself

assign a semantic meaning to each feature.

A causal abstraction requires an additional model of the proposed high-level computation and a correspondence between interventions [21]. For deterministic models with well-defined outcomes, the exact agreement condition can be written

$$\tau(M_{\text{low}}(\xi;\iota)) = M_{\text{high}}(\tau_C(\xi);\omega(\iota))\,. \tag{A.10}$$

ξ	input or background context
ι	an allowed low-level intervention
$M_{\text{low}}(\xi;\iota)$	low-level variable assignment under that intervention
M_{high}	proposed high-level causal model
τ	map from low-level to high-level variable assignments
τ_C	corresponding map of background contexts
ω	map from low-level to high-level interventions

The equality must hold over the specified admissible contexts and interventions for exact abstraction; finite experiments can assess agreement on the cases tested. A sparse representation alone does not supply the high-level model or these maps. The thesis's corrected feature-removal result is not a demonstration of (A.10).

REFERENCES

Bibliography

[1] Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 2(11), 559–572.

[2] Hotelling, H. (1933). Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6), 417–441.

[3] Jolliffe, I. T. (1986). *Principal Component Analysis*. Springer Series in Statistics. Springer, New York. https://doi.org/10.1007/978-1-4757-1904-8

[4] Software Engineering Institute. (2016). *Insider Threat Test Dataset*. Carnegie Mellon University Software Engineering Institute. https://doi.org/10.1184/R1/12841247.v1

[5] Glasser, J., & Lindauer, B. (2013). Bridging the gap: A pragmatic approach to generating insider threat data. In *2013 IEEE Security and Privacy Workshops* (pp. 98–104). IEEE. https://doi.org/10.1109/SPW.2013.37

[6] Le, D. C., Zincir-Heywood, N., & Heywood, M. I. (2020). Analyzing data granularity levels for insider threat detection using machine learning. *IEEE Transactions on Network and Service Management*, 17(1), 30–44. https://doi.org/10.1109/TNSM.2020.2967721

[7] Shlens, J. (2014). *A Tutorial on Principal Component Analysis*. arXiv. https://doi.org/10.48550/arXiv.1404.1100

[8] Cappelli, D., Moore, A., & Trzeciak, R. (2012). *The CERT Guide to Insider Threats: How to Prevent, Detect, and Respond to Information Technology Crimes (Theft, Sabotage, Fraud)*. Addison-Wesley Professional.

[9] HPC Wiki, The University of Southern Mississippi. (2024). *Main Page*. Magnolia HPC Wiki. Suggested infrastructure acknowledgment: The authors acknowledge HPC at The University of Southern Mississippi supported by the National Science Foundation under the Major Research Instrumentation (MRI) program via Grant # ACI 1626217. `http://magnolia.usm.edu/index.php?title=Main_Page&oldid=1116`

[10] Du, M., Li, F., Zheng, G., & Srikumar, V. (2017). DeepLog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*.

[11] Tuor, A., Kaplan, S., Hutchinson, B., Nichols, N., & Robinson, S. (2017). Deep learning for unsupervised insider threat detection in structured cybersecurity data streams. In *AAAI Workshop on Artificial Intelligence for Cyber Security*.

[12] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*.

[13] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

[14] Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*.

[15] Qwen Team. (2025). *Qwen3 technical report*. arXiv preprint arXiv:2505.09388.

[16] Qwen Team. (2025). *Qwen3-8B model configuration*. https://huggingface.co/Qwen/Qwen3-8B/blob/main/config.json. Accessed October 14, 2026.

[17] Bricken, T., Templeton, A., Batson, J., Chen, B., Jermyn, A., Conerly, T., Turner, N. L., et al. (2023). Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*. https://transformer-circuits.pub/2023/monosemantic-features

[18] Cunningham, H., Ewart, A., Riggs, L., Huben, R., & Sharkey, L. (2023). Sparse autoencoders find highly interpretable features in language models. arXiv preprint arXiv:2309.08600.

[19] Shannon, C. E. (1948). A mathematical theory of communication. *The Bell System Technical Journal*, 27(3), 379–423.

[20] Gao, L., Dupré la Tour, T., Tillman, H., Goh, G., Troll, R., Radford, A., Sutskever, I., Leike, J., & Wu, J. (2024). Scaling and evaluating sparse autoencoders. arXiv preprint arXiv:2406.04093.

[21] Geiger, A., Ibeling, D., Zur, A., Chaudhary, M., Chauhan, S., Huang, J., Arora, A., Wu, Z., Goodman, N., Potts, C., & Icard, T. (2025). Causal abstraction: A theoretical foundation for mechanistic interpretability. *Journal of Machine Learning Research*, 26(83), 1–64. https://jmlr.org/papers/v26/23-0058.html